CopyTrace: Formula Reference

Digital image copy-move forgery detection and localization using DCT, ORB, geometric verification, and morphology.

4. Step-by-step method and formulas

4.1 Read, grayscale, and resize

The program keeps the color photo for the final overlay and makes a grayscale copy for matching. A standard grayscale conversion is:

$$Y(x,y) = 0.299R(x,y) + 0.587G(x,y) + 0.114B(x,y)$$

Here, R, G, and B are the red, green, and blue pixel values, and Y is brightness. OpenCV reads color channels in BGR order but applies the corresponding weights to the correct channels.

For an image of height H and width W, the resize scale is:

$$s = \min\left(1, \frac{768}{\max(H,W)}\right)$$

The image is resized only when its longest side is larger than 768 pixels. Both dimensions are multiplied by s, preserving the aspect ratio. The report records the original and processed dimensions. Masks and boxes use processed-image coordinates.

4.2 DCT block matching

DCT means Discrete Cosine Transform. It describes the brightness patterns inside a block using frequency values. Put simply, it turns the texture of a small patch into a short list of numbers.

The program scans the image with overlapping 16 by 16 pixel blocks, moving 4 pixels at a time. Flat blocks are skipped because plain sky or walls do not provide distinctive patterns.

For image width W and height H, the number of block positions is:

$$n_x = \left\lfloor \frac{W-16}{4} \right\rfloor + 1, \qquad n_y = \left\lfloor \frac{H-16}{4} \right\rfloor + 1$$

$$n_{\text{blocks}} = n_x n_y$$

For a block X(x,y) of size N by N, with N = 16, the two-dimensional DCT is:

$$F(u,v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} X(x,y) \cos\left(\frac{\pi(2x+1)u}{2N}\right) \cos\left(\frac{\pi(2y+1)v}{2N}\right)$$

The scale factor is:

$$\alpha(k) = \begin{cases} \sqrt{\frac{1}{N}}, & k = 0, \\ \sqrt{\frac{2}{N}}, & k > 0. \end{cases}$$

The DC coefficient F(0,0) mostly represents average brightness. The detector skips it and keeps 15 low-frequency AC coefficients in zigzag order. Low frequencies describe broad patterns and tend to be less sensitive to small pixel changes than fine details.

Let the selected coefficients be the vector c:

$$\mathbf{c} = [c_1, c_2, \ldots, c_{15}]$$

The vector is normalized so overall contrast has less effect:

$$\mathbf{d} = \frac{\mathbf{c}}{\|\mathbf{c}\|_2}, \qquad \|\mathbf{c}\|_2 = \sqrt{\sum_{i=1}^{15} c_i^2}$$

Blocks with very small descriptor energy are skipped. The distance between two normalized descriptors is:

$$D(\mathbf{d}_a, \mathbf{d}_b) = \sqrt{\sum_{i=1}^{15} (d_{a,i} - d_{b,i})^2}$$

A smaller distance means the blocks are more alike. The implementation accepts distances below 0.30 and ignores blocks closer than 32 pixels.

If block A is at (x_a,y_a) and block B is at (x_b,y_b), their displacement is:

$$\Delta = (x_b - x_a,\ y_b - y_a)$$

Displacements are grouped on an 8-pixel grid:

$$g = \operatorname{round}\left(\frac{\Delta}{8}\right)$$

A likely copy should have many similar block pairs with about the same movement. The program requires at least 12 distinct pairs. They must form a connected patch and cover at least 20 pixels horizontally and vertically at both ends. This helps reject scattered coincidences.

4.3 ORB feature matching

ORB means Oriented FAST and Rotated BRIEF. It finds distinctive points, often corners and textured spots, and describes the nearby image pattern with binary values. ORB can help with some rotated or resized copies.

FAST looks at pixels around a candidate point to decide whether the center resembles a corner. ORB estimates the point's direction using image-intensity moments:

$$m_{pq} = \sum_{x,y} x^p y^q I(x, y)$$

$$\theta = \operatorname{atan2}(m_{01}, m_{10})$$

BRIEF compares pairs of pixel intensities around the point. Each comparison becomes one bit:

$$b_k = \begin{cases} 1, & I(T_\theta(a_k)) < I(T_\theta(b_k)), \\ 0, & \text{otherwise.} \end{cases}$$

Here, a_k and b_k are sample locations and T_theta rotates the locations to follow the point's direction. Descriptors are compared by Hamming distance:

$$D_H(\mathbf{a}, \mathbf{b}) = \sum_{k=1}^{K} [a_k \neq b_k]$$

Each bracket is 1 if the bits differ and 0 if they are equal. The C++ program requests up to 2,500 ORB points; MATLAB keeps up to 900. Both reject nearby, ambiguous, and duplicate matches. C++ uses a nearest-match ratio check. If d_1 is the closest match distance and d_2 the second closest, it keeps a match when:

$$d_1 < 0.8 d_2, \qquad d_1 \leq 48$$

A pair must also select each other as best matches and be at least 48 pixels apart. Matches in each 8-pixel cell are reduced to avoid counting the same small area many times.

4.4 Geometric verification

Two unrelated corners can look similar by chance. The program checks whether many matches follow one common transformation. The similarity model includes rotation, scale, and translation:

$$x' = s(x \cos\theta - y \sin\theta) + t_x$$

$$y' = s(x \sin\theta + y \cos\theta) + t_y$$

Here, s is scale, theta is rotation, and t_x and t_y are horizontal and vertical movement. C++ uses RANSAC; MATLAB uses the related MSAC method. Both estimate a transformation and identify matches that agree with it.

The reprojection error for point pair i is:

$$e_i = \|\mathbf{q}_i - T(\mathbf{p}_i)\|_2$$

A point pair is counted as an inlier when its error is at most 4 pixels. The program requires at least 8 inliers, a scale between 0.55 and 1.8, and points spread over an area rather than in a straight line or tiny cluster. The source and target point groups each need a 20-pixel span and a convex hull area of at least 400 square pixels.

4.5 Combine and clean the masks

DCT marks the accepted 16 by 16 blocks at both matching locations. ORB marks a disk of radius 13 pixels around each verified point. The masks are combined with a logical OR:

$$M_0(x,y) = M_{\mathrm{DCT}}(x,y) \vee M_{\mathrm{ORB}}(x,y)$$

Morphology cleans the combined mask. Dilation expands foreground pixels; erosion shrinks them. For foreground set A and structuring shape B:

$$A \oplus B = \{z : (\hat{B})_z \cap A \neq \varnothing\}$$

$$A \ominus B = \{z : B_z \subseteq A\}$$

Opening removes small specks; closing joins small gaps:

$$A \circ B = (A \ominus B) \oplus B$$

$$A \bullet B = (A \oplus B) \ominus B$$

The program applies closing, then opening, then removes connected regions smaller than 180 pixels. MATLAB uses disk-shaped structuring elements; C++ uses elliptical elements. Each remaining connected component becomes a region with a bounding box.

At flagged pixels, the output blends the original pixel I with red R_c:

$$I_{\mathrm{out}}(x,y) = 0.55 I(x,y) + 0.45 R_c \qquad \text{when } M(x,y) = 1$$

The program draws a yellow box around each region. These boxes are approximate guides, not exact object outlines.

5. Evaluation formulas

When a matching truth mask is supplied, compare the prediction with the truth one pixel at a time:

- TP (true positive): both the prediction and truth mark the pixel.
- FP (false positive): the prediction marks it, but truth does not.
- FN (false negative): truth marks it, but the prediction does not.
- TN (true negative): neither marks the pixel.

The report calculates precision, recall, F1, and intersection over union (IoU):

$$\mathrm{Precision} = \frac{TP}{TP + FP}$$

$$\mathrm{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2\,\mathrm{Precision}\,\mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}} = \frac{2TP}{2TP + FP + FN}$$

$$\mathrm{IoU} = \frac{TP}{TP + FP + FN}$$

The program returns 0 if a score's denominator is zero. The truth mask should match the original photo dimensions and mark both the source and pasted area. If it labels only the destination, scores will not fairly compare against a prediction that may include both regions.

The flagged percentage is:

$$\mathrm{FlaggedArea}(\%) = 100 \frac{\sum_{x=1}^{W} \sum_{y=1}^{H} M(x, y)}{WH}$$

Without a truth mask, the program can show suspicious regions and flagged area, but cannot calculate precision, recall, F1, or IoU. A collection with authentic photos is also needed to calculate image-level false-positive rate:

$$\mathrm{FalsePositiveRate} = \frac{\text{authentic photos flagged}}{\text{authentic photos tested}}$$